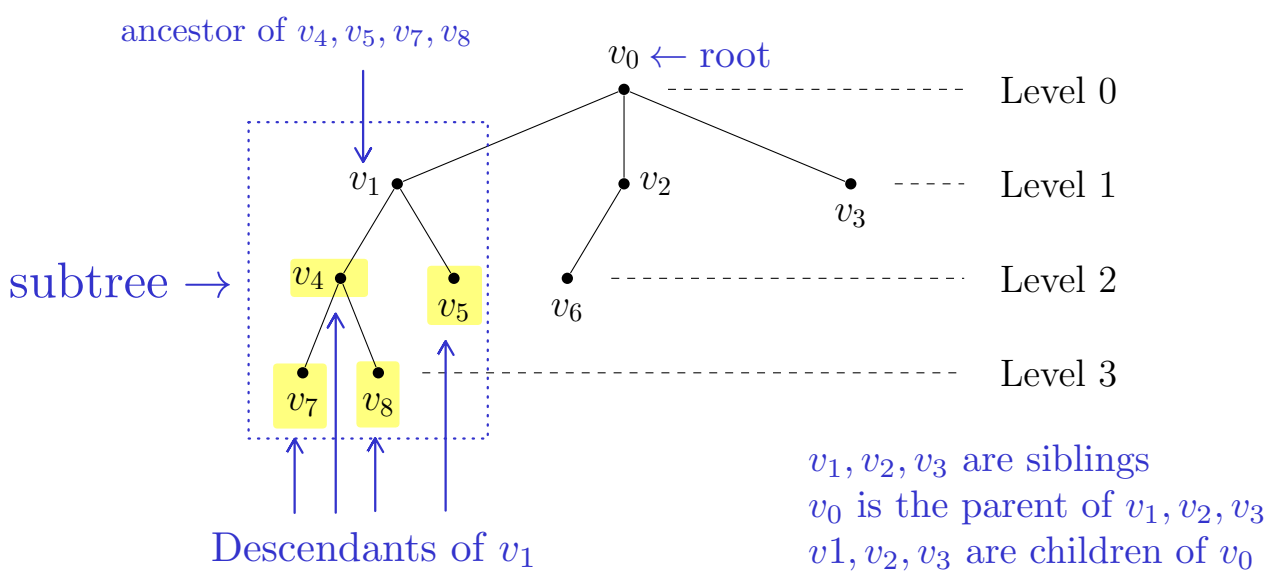
10.5: Rooted Trees

Definition.

- A **rooted tree** is a tree in which there is one vertex that is distinguished from the others and is called the **root**.
- The **level** of a vertex is the number of edges along the unique path between it and the root.
- The **height** of a rooted tree is the maximum level of any vertex of the tree.
- Given the root or any internal vertex v of a rooted tree, the **children** of v are all those vertices that are adjacent to v and are one level farther away from the root than v.
- If w is a child of v, then v is called the **parent** of w, and two distinct vertices that are both children of the same parent are called **siblings**.
- Given two distinct vertices v and w, if v lies on the unique path between w and the root, then v is an **ancestor** of w and w is a **descendant** of v.



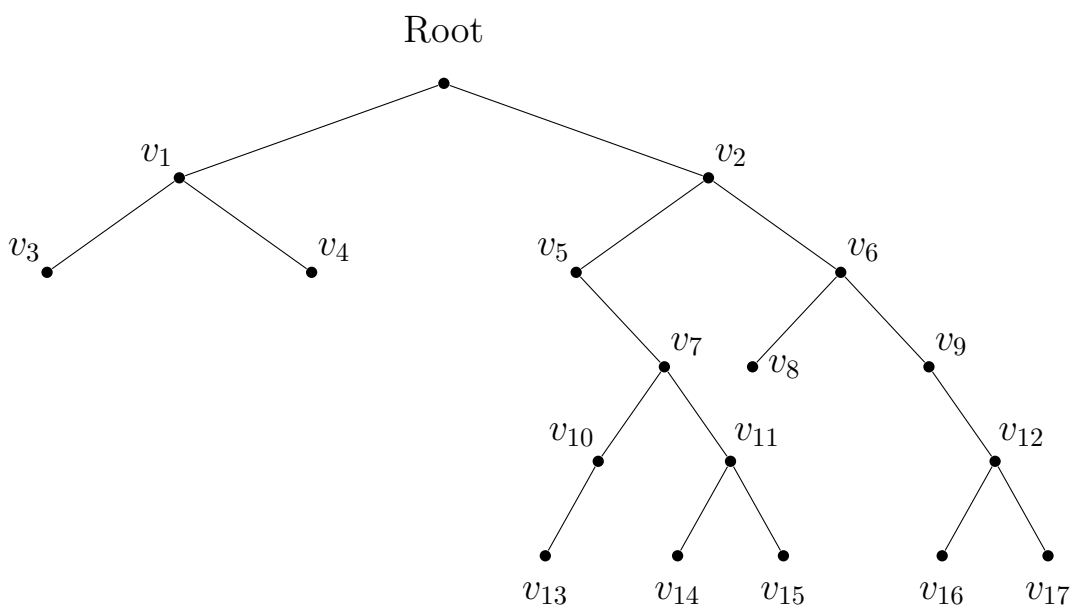
Definition.

- A **binary tree** is a rooted tree in which every parent has at most two children.
- Each child in a binary tree is designated either a **left child** or a **right child** (but not both), and every parent has at most one left child and one right child.
- A **full binary tree** is a binary tree in which each parent has exactly two children.

Given any parent v in a binary tree T, if v has a left child, then the **left subtree** of v is the binary tree

- whose root is the left child of v,
- whose vertices consist of the left child of v and all its descendants, and
- whose edges consist of all those edges of T that connect the vertices of the left subtree.

The **right subtree** of v is defined analogously.



Theorem 10.5.1
If k is a positive integer and T is a full binary tree with k internal vertices, then

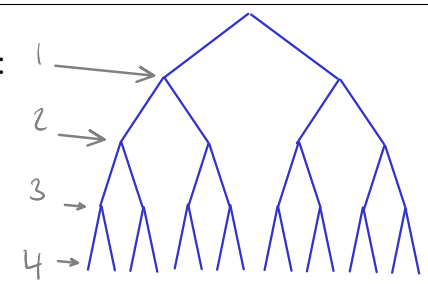
- T has a total of $2k+1$ vertices, and
- T has $k+1$ leaves.

Theorem 10.5.2
For ever integer $h \geq 0$, if T is any binary tree with height h and t leaves, then

$$t \leq 2^h \Leftrightarrow \log_2(t) \leq h.$$

Corollary 10.5.3
A full binary tree of height h has 2^h leaves.

Example. Determine if the following binary trees can exist:



Binary tree, height 4, eight leaves

Yes; $h=4$, $t=8$ $\quad 8 \leq 2^4 = 16$

Full binary tree, nine vertices, five internal vertices

No; $k=5$ internal vertices

Theorem 10.5.1 $\Rightarrow$ $2k+1 = 2(5)+1 = 11$ vertices $\neq 9$ vertices

Full binary tree, four internal vertices

No; $k=4$ $\quad$ Theorem 10.5.1 $\Rightarrow$ $2k+1 = 2(4)+1 = 9$ vertices

Corollary 10.5.3 $\Rightarrow$ 2^h leaves, but $9 \neq 2^h$

Full binary tree, height 4, eighteen leaves

No; $h=4$ $\quad$ Corollary 10.5.3 $\Rightarrow$ $2^h = 2^4 = 16$ leaves $\neq 18$ leaves

Definition.

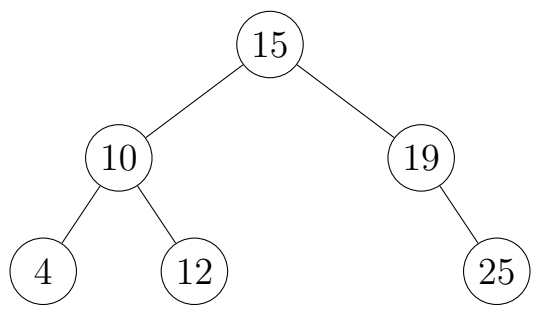
- A **binary search tree** is a binary tree in which data records can be stored, searched, and processed.
- Each element is assigned a **key**, which is used to order the elements.

In a binary tree, for every internal vertex v:

- all the keys in the left subtree of v are less than the key in v, and
- all the keys in the right subtree of v are greater than the key in v.

Example. The following is a binary search tree for the following keys:

$$15, 10, 19, 25, 12, 4$$



Example. Build the binary search tree for the keys $19, 10, 25, 4, 15, 12$

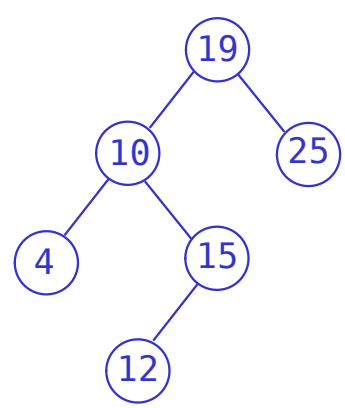
$10 < 19 \to$ left child

$25 > 19 \to$ Right child

$4 < 19$, $4 < 10 \to$ left child

$15 < 19$, $15 > 10 \to$ right child

$12 < 19$, $12 > 10$, $12 < 15 \to$ left child



Algorithm 2: Algorithm 10.5.1: Building a Binary Search Tree

```
Input  : A totally ordered, nonempty set K of keys
Initialize T to have one vertex, the root, and no edges
Choose a key from K to insert into the root
while (there are still keys to be added) do
    Choose a key, newkey, from K to add.
    Let the root be called v, let key(v) be the key at the root
    success := 0
    while success == 0 do
        if newkey < key(v) then
            if v has a left child then
                v_L :=the left child
                v   := v_L
            else
                add a vertex v_L to T as the left child for v
                add an edge to T to join v to v_L
                insert newkey as the key for v_L
                success := 1
            end
        if newkey > key(v) then
            if v has a right child then
                v_R :=the right child
                v   := v_R
            else
                add a vertex v_R to T as the right child for v
                add an edge to T to join v to v_R
                insert newkey as the key for v_R
                success := 1
            end
    end
end
Output: A binary search tree T for the set K of keys
```